

3. Software Requirements Specification (SRS)

The Software Requirements Specification (SRS) document defines what the E-Commerce Multi-Vendor Platform must do (functional requirements) and how well it must do it (non-functional requirements). This chapter serves as the agreement between the developer and the expected system behavior.

3.1 Functional Requirements

Functional requirements describe the core features and behaviors the system must support. These are organized by module and associated with specific user roles (Admin, Vendor, Customer, or Public).

FR-1: User Management

ID	Requirement Description	Associated Role
FR-1.1	A new user shall be able to register by providing a username, email, password, and role (Customer or Vendor).	Public / Guest
FR-1.2	A registered user shall be able to log in using their email and password and receive a JWT access token and refresh token.	Public / Guest
FR-1.3	The system shall automatically redirect the user to their role-specific dashboard after successful login (Customer → Products, Vendor → Vendor	All Roles

	Dashboard, Admin → Admin Dashboard).	
FR-1.4	The system shall allow a logged-in user to view their own profile details (username, email, role).	All Roles
FR-1.5	The system shall refresh the JWT access token automatically using the refresh token when the access token expires, without requiring the user to log in again.	All Roles
FR-1.6	A logged-in user shall be able to log out, which clears their stored tokens from the browser.	All Roles
FR-1.7	The Admin shall be able to view a list of all registered users on the platform.	Admin
FR-1.8	A logged-in user shall be able to view and update their own profile (username, email, phone, address). Vendors can also update their shop name and description from the Profile page.	All Roles

FR-2: Vendor Management

ID	Requirement Description	Associated Role
----	-------------------------	-----------------

FR-2.1	A user registered as a Vendor shall have a Vendor profile created with a shop name and description.	Vendor
FR-2.2	A newly registered Vendor account shall be in "Pending Approval" status by default and cannot add products until approved.	Vendor
FR-2.3	The Admin shall be able to view all vendor registrations with their approval status.	Admin
FR-2.4	The Admin shall be able to approve a pending vendor, enabling them to list products.	Admin
FR-2.5	The Admin shall be able to revoke an approved vendor's access.	Admin
FR-2.6	The Admin shall be able to manage customer accounts and view all users separately categorized by their role (Admins, Vendors, Customers).	Admin

FR-3: Product Management

ID	Requirement Description	Associated Role
FR-3.1	An approved Vendor shall be able to add a new product with name,	Vendor (Approved)

	description, price, stock quantity, and category.	
FR-3.2	A Vendor shall be able to edit the details of their own products.	Vendor (Owner)
FR-3.3	A Vendor shall be able to delete their own products.	Vendor (Owner)
FR-3.4	A Vendor shall not be able to edit or delete products belonging to other vendors.	Vendor (System Rule)
FR-3.5	All users (including guests) shall be able to view the full list of active products.	Public / Guest
FR-3.6	The Admin shall be able to create and manage product categories (name and slug).	Admin
FR-3.7	Any user (including guests) shall be able to view a public Vendors/Stores listing page showing all approved vendor shops.	Public / Guest
FR-3.8	An approved Vendor shall be able to upload one or more product images when creating or updating a product. Images are stored in the server filesystem and served via Django's media URL.	Vendor (Approved)
FR-3.9	The Admin shall be able to view and manage all products listed on the	Admin

	platform via a dedicated Admin Products page.	
--	---	--

FR-4: Search and Filter

ID	Requirement Description	Associated Role
FR-4.1	Any user shall be able to search products by keyword in the product name.	Public / Guest
FR-4.2	Any user shall be able to filter products by category using a dropdown selector.	Public / Guest
FR-4.3	Search and filter results shall update dynamically without a full page reload.	Public / Guest
FR-4.4	A user shall be able to clear applied search and filter to return to the full product listing.	Public / Guest

FR-5: Cart Management

ID	Requirement Description	Associated Role
FR-5.1	A logged-in Customer shall be able to add a product to their shopping cart.	Customer
FR-5.2	A Customer shall be able to increase or decrease the quantity of any item in the cart.	Customer

FR-5.3	A Customer shall be able to remove an individual item from the cart.	Customer
FR-5.4	A Customer shall be able to clear the entire cart in one action.	Customer
FR-5.5	The cart shall display a running total of all items and quantities.	Customer
FR-5.6	The cart item count shall be visible in the navigation bar at all times.	Customer

FR-6: Order Management

ID	Requirement Description	Associated Role
FR-6.1	A Customer shall be able to place an order from the checkout page by providing a delivery address.	Customer
FR-6.2	On successful order placement, the system shall deduct the ordered quantity from the product's stock.	System
FR-6.3	A Customer shall be able to view their complete order history with order ID, date, total, and status.	Customer
FR-6.4	A Customer shall be able to expand an order to see	Customer

	individual items, quantities, and prices.	
FR-6.5	A Vendor shall be able to view all orders that contain their products.	Vendor
FR-6.6	A Vendor shall be able to update the status of an order from Pending → Shipped → Delivered.	Vendor
FR-6.7	The Admin shall be able to view all orders across all vendors on the platform.	Admin

FR-7: Dashboard

ID	Requirement Description	Associated Role
FR-7.1	The Customer dashboard shall display total orders placed, pending orders, delivered orders, and total amount spent.	Customer
FR-7.2	The Vendor dashboard shall display total products listed, total orders received, pending orders, and total revenue from delivered orders.	Vendor
FR-7.3	The Admin dashboard shall display total users, total vendors, total customers, total products, total orders, pending orders, and	Admin

	platform-wide revenue from delivered orders.	
FR-7.4	All dashboard statistics shall be fetched in real-time from the <code>/api/dashboard/stats/</code> API endpoint, which returns different data payloads based on the authenticated user's role.	All Roles

3.2 Non-Functional Requirements

Non-functional requirements define the quality standards, architectural constraints, and security thresholds the system must satisfy beyond its basic features.

NFR-1: Security

ID	Requirement Description
NFR-1.1	All API endpoints except registration, login, and product listing shall require a valid JWT access token in the <code>Authorization: Bearer <token></code> header.
NFR-1.2	Role-based access control (RBAC) shall be enforced on the server side — a Customer cannot access Vendor or Admin endpoints even if they manually construct and send HTTP requests.
NFR-1.3	Vendors shall only be able to modify or delete their own products — object-level permission checks shall be strictly enforced in views and viewsets.
NFR-1.4	Passwords shall be stored as salted cryptographic hashes using Django's default PBKDF2 hashing algorithm — never in plain text.

NFR-1.5	JWT access tokens shall expire after 60 minutes, while refresh tokens shall expire after 7 days to maintain a secure balance between security and user convenience.
NFR-1.6	CORS (Cross-Origin Resource Sharing) shall be configured to only allow requests from the trusted React frontend origin (http://localhost:5173) and block unknown clients.

NFR-2: Performance

ID	Requirement Description
NFR-2.1	API responses for standard product listing and order queries shall complete within 500 milliseconds under normal usage conditions.
NFR-2.2	The React frontend initial page load, including stylesheet download and main JS bundling, shall complete within 3 seconds on a standard broadband connection.
NFR-2.3	Search and filter operations on the products page shall process client-side or return server results within 300 milliseconds to avoid lagging interface feedback.
NFR-2.4	Database queries shall leverage Django ORM's <code>select_related</code> and <code>prefetch_related</code> methods where applicable to avoid N+1 query bottlenecks.

NFR-3: Usability

ID	Requirement Description
----	-------------------------

NFR-3.1	The interface shall be clean and intuitive — a new user shall be able to register, browse products, and place an order without external documentation or assistance.
NFR-3.2	All forms (registration, login, product listings) shall validate fields dynamically and display clear, descriptive errors (e.g., "Email already registered" instead of "Database Error").
NFR-3.3	The navigation bar shall dynamically adapt to display only the menu links relevant to the current user's role.
NFR-3.4	The platform layout shall be responsive, adjusting for tablet, desktop, and mobile screen resolutions.
NFR-3.5	Active loading indicators shall be displayed during asynchronous REST API requests so the user is informed of pending operations.

NFR-4: Scalability

ID	Requirement Description
NFR-4.1	The Django backend API shall remain stateless, avoiding standard server-side sessions, allowing horizontal scaling behind load balancers.
NFR-4.2	The database layer shall remain modular — swapping the development SQLite database for production PostgreSQL must require changing only the <code>DATABASES</code> config dictionary in <code>settings.py</code> .

NFR-4.3	The independent frontend and backend services shall allow decoupled deployment, scaling, and codebase updates.
NFR-4.4	New user roles or vendor permissions can be added to the platform in the future without modification of the base JWT auth architecture.

NFR-5: Reliability

ID	Requirement Description
NFR-5.1	Order placement operations shall utilize database atomic transactions (<code>transaction.atomic()</code>) to guarantee complete execution of stock deduction and order item creation, preventing database corruption on system interruptions.
NFR-5.2	All backend endpoints shall return precise standard HTTP status codes (200, 201, 400, 401, 403, 404, 500) alongside structured JSON payloads.
NFR-5.3	The frontend Axios client shall intercept API errors and show notification messages rather than crashing or freezing the application state.

NFR-6: Maintainability

ID	Requirement Description
NFR-6.1	The Django backend shall be split into modular apps (<code>users</code> , <code>vendors</code> , <code>products</code> , <code>orders</code> , <code>dashboard</code>) with isolated models, views, and urls.

NFR-6.2	The React frontend shall follow component-based architecture — separating shared reusable UI components (Navbar , ProductCard) from container pages.
NFR-6.3	All API calls in the frontend shall route through a centralized Axios service configuration (src/services/api.js) to maintain authorization headers in a single location.
NFR-6.4	Source files shall adhere to platform style guide conventions: PEP 8 guidelines for Python/Django and ESLint rules for JavaScript/React.